

Parallel optimization of the DisloCluster coupled march

Five fixes, three falsified hypotheses, two retractions, a bit-identical $1.92\times$ at 200 nm, and $3.1\times$ on the production case

DisloCluster development note

2 September 2026

Abstract

The coupled march spends its time in two solvers: a *slow* step integrating independent stiff ODEs at every mesh node (ZrMicro, OpenMP batch) and a *fast* step solving a steady reaction–diffusion FE problem (MoDELib3 DDomp). Neither was using the machine, and in *every* case the reason was not the one the phase labels suggested. Five changes follow from measurement — sharding the slow batch across processes, removing a token round trip that consumed 47 % of the slow step, caching the FE sparsity pattern across Newton iterations, correcting the fast step’s thread count, and making DDomp resident so it stops factorizing the same matrix 64 times — and every one is verified **bit-identical** against the unoptimized code. On a 200 nm hexagonal march to 1 dpa the wall time falls $161.7\text{ s} \rightarrow 84.1\text{ s}$, a factor **1.92**, with the slow step $2.39\times$ and a steady-state fast solve $2.32\times$. On the production case — a $1000\text{ nm} \times 1600\text{ nm}$ hexagonal prism, 189 533 CD nodes, 16 doses to 40 dpa — the gain is larger, because the costs removed are the ones that grow fastest with N : matched intervals run $3439\text{ s} \rightarrow 1067\text{ s}$, a factor **3.22**, and the march itself $15.36\text{ h} \rightarrow 4.89\text{ h}$, a factor **3.14** (§6). Two claims made in earlier revisions of this note were wrong and are retracted in §3.1 and §4; both were corrected by profiling rather than by argument, which is the main methodological point of the document.

1 The machine, and why it matters

All measurements are on a dual-socket Intel Xeon Gold 6230: **40 physical cores, 80 logical threads**, Windows 10, with the 3-D solver under WSL2.

Windows partitions 80 logical processors into *two processor groups* of 40, and a process is confined to one unless it calls the group-affinity API:

```
System.Environment.ProcessorCount -> 40 (not 80)
process affinity mask -> 1099511627775 =  $2^{40} - 1$ 
```

So one process can address at most half the machine, and asking for 80 threads oversubscribes 40 logical processors two-to-one.

2 The slow step

2.1 Thread scaling: measured

The slow step is embarrassingly parallel by construction — freezing the mobile species removes the only spatial coupling, leaving N independent 19-dimensional systems with no neighbour state. Threads stopped paying almost immediately. Fixed work of 20 000 cases, three passes, machine idle:

processes	threads	total	cases/s	speedup
1	1	1	1 544	1.00×
1	8	8	4 541	2.94×
1	40	40	2 678	1.73×
1	80	80	1 130	0.73×
5	8	40	14 347	9.29×
20	4	80	20 438	13.23×
40	2	80	18 469	11.96×

At its configured 80 threads the batch ran *slower than one thread*, and the same 80 threads arranged as 20 processes of 4 are 18× apart.

2.2 Three hypotheses, all falsified

Before changing anything, the obvious mechanisms were built and measured. All are **ruled out**, and recording that is the point of this section:

Hypothesis	Test	Result
False sharing on <code>status[c]</code> , <code>outbuf[c]</code>	rebuild with <code>schedule(dynamic, 64)</code>	1.00–1.01×
SUNDIALS error-handler lock	remove all 20 000 messages at source	scaling unchanged
Progress text written to the pipe	measure the volume	0.69 MB/call, negligible

The surviving inference is from the data: *processes scale and threads do not*, implicating a resource private to a process and shared by its threads. The per-process heap fits (every case builds an `ostream` and a `string`), but this is **not demonstrated** and is a hypothesis only.

2.3 Fix 1: sharding

Batching merges work to amortize per-call overhead; *sharding* splits it to escape a per-worker ceiling. The code already batched — all ~190 000 cases into one process, to amortize the Windows-dominant process-spawn cost — which is correct but does not say when to stop. `run_cpp_solver_batch` now splits across concurrent processes, each a self-contained batch file with its own `@BASE` header, results scattered back by original index. Each shard is still a batch; `_SHARD_MIN_CASES` is where the two arguments balance.

At 189 533 nodes: 52.3 s for the best single process against 29.2 s sharded, **1.79×**, bit-identical.

2.4 Fix 2: the token round trip

With the solver ceiling removed, 47 % of the slow step was Python string handling:

phase	before	%	after
dedup	0.61 s	1.5	0.72 s
build_immobile_cases	5.31 s	13.2	0.00 s
_batch_lines	13.64 s	33.9	2.53 s
solver + parse	20.68 s	51.4	11.46 s
TOTAL	39.94 s		14.72 s 2.71×

Every case is the same ~ 87 -token base with 19 `y0` slots overwritten. The builder materialized all 87 per case; `_batch_lines` parsed them back into dictionaries and diffed 157 204 of them to rediscover the split the builder already knew. Roughly 78% of the tokens existed only to be discarded. `ImmobileCases` now holds the base and the `y0` array and formats only what varies.

Key order is the protocol. `collect_solver_args` *already* emits `y0_0..y0_18`, so a case dictionary overwrites them **in place**: the order is the base’s, not base-then-`y0`. A first attempt guarded against that and fell back — silently doing nothing, and profiling identically. Byte-identity is asserted at $n = 1, 2, 500, 5000$, on an all-identical batch (the empty-delta guard) and on a sliced shard.

3 The fast step

3.1 Per-phase scaling, and the first retraction

Whole-solver thread scaling is poor ($1.68\times$ from 1 to 40 threads at 200 nm, a 7% spread at 1000 nm) and worst at the inherited default of 80. Resolving it *by phase* settles where the serial time is:

threads	assembly	“precond”	krylov	total
1	6.03	3.48	6.97	16.52
4	4.14	3.45	3.67	11.30
20	3.72	3.83	2.63	10.23
scaling	$1.62\times$	$0.91\times$	$2.65\times$	$1.61\times$

Retraction 1. An earlier revision asserted FE assembly was serial and made “parallelize assembly” priority 2. **Wrong.** `BilinearWeakForm::globalTriplets` is already parallel, over explicit contiguous chunks concatenated in chunk order precisely so the triplet sequence stays bit-identical to the serial one. It scales poorly ($1.62\times$), but poor is not absent. Krylov, expected to be the bottleneck, is the *best*-scaling phase.

3.2 Fix 3: cache the sparsity pattern

The dominant phase is not preconditioning: the reaction solver uses `use_directSolver = false`, so its preconditioner is Eigen’s diagonal one. The timer brackets `computeFromTriplets`, i.e. sparse-matrix *construction* — `setFromTriplets` on A , a pass to strike out the Dirichlet rows and columns, then `setFromTriplets` on A_1 , twice per Newton iteration and serial in Eigen.

Across the Newton loop that pattern is invariant; only values move. The first call records `aSlot` (triplet $\rightarrow$ slot in A) and `a1FromA` (slot in $A \rightarrow$ slot in A_1) by binary search inside each compressed row, $O(\text{nnz} \log \text{nnz})$ once; later calls zero the value arrays and accumulate in place, in triplet order — the order `setFromTriplets` sums duplicates in — so the matrix is bit-identical.

200 nm, per fast solve	baseline	pattern cache	gain
“precond”	4.758 s	2.885 s	$1.65\times$
solve total	10.378 s	8.095 s	$1.28\times$

Convergence unchanged to every printed digit (4 iterations, $c_{\text{Error}} = 8.785110566792336 \times 10^{-7}$). At 1000 nm the invocation falls 476.0 s $\rightarrow$ 415.6 s.

3.3 Fix 4: stop oversubscribing

`qssa.py` launched `DDomp` through `wsl.exe` with no environment, so it inherited 80 threads. It now sets `OMP_NUM_THREADS=20` and forwards it via `WSLENV`, which WSL does not otherwise inherit.

4 Profiling the invocation, and the second retraction

Half the fast step lay outside the stage timers, and identifying it required instrumenting `initializeSolver`, whose cost no existing timer covered. Per invocation:

phase	200 nm	1000 nm	ratio	N^k	% wall (1000 nm)
mobile stages (the solve)	8.35	156.32	18.7	1.33	39.2 %
Cholesky of diffusion op.	2.92	175.60	60.2	1.86	44.1 %
unexplained in-solve	0.26	28.96	111.4	2.14	7.3 %
process startup + WSL	2.16	22.80	10.6	1.07	5.7 %
write <code>evl_0</code>	1.22	14.73	12.1	1.14	3.7 %
mesh read + create + faces	0.41	12.04	29.4	1.54	3.0 %
read <code>evl_0</code>	0.37	2.91	7.9	0.94	0.7 %
TrialFunction construction	0.42	2.09	4.9	0.72	0.5 %
TOTAL WALL	14.92	398.51	26.7	1.49	

Retraction 2. Earlier revisions attributed the gap to “mesh reading, trial-function construction, FE data structures and `evl` output.” **Mesh handling is 3 %.** The cost is `mSolver.compute(mBWF)` — a `CholmodSupernodalLLT` factorization of the diffusion operator, **larger than the entire iterative solve** and growing as $N^{1.86}$ against the solve’s $N^{1.33}$, so its share rises with every increase in domain size. Its companion in `initializeSolver`, the Dirichlet BC assignment, is 0.014 s — nothing.

It is also performed to serve *one* diffusion solve of 0.93 s, on a matrix that depends only on the mesh and the diffusion tensor and is therefore identical across all 64 invocations of a 40 dpa march.

4.1 Fix 5: a resident `DDomp`

`DDomp` gains an opt-in `--server` mode: mesh, trial functions, `DefectiveCrystal` and the factorization are built once, and each `SOLVE` line on stdin re-reads the configuration and re-solves. `initializeConfiguration` assigns only the CD fields, and `initializeSolver` is guarded by `solverInitialized`, so the numerics are untouched. Without `--server` the path is byte-for-byte the original; `DISLOCLUSTER.DDOMP_SERVER=0` restores it from the driver side.

Two pieces of carried state, and why they matter more than the speedup. A resident solver is only worth having if it is exactly equivalent, and two things were not:

1. `runSteps()` advances `runID` and does `totalTime += dt`, with `ClusterDynamics` taking `lastUpdateTime` from `totalTime`. Restoring `runID` alone leaves cycle 2 doing nothing and silently returning cycle 1’s answer.
2. `mobileClustersIncrement` is zeroed in the **constructor only**, and the first Newton assembly of every cycle reads it (`bWF_R1c`, `bWF_R1c`). Left over, it perturbed the result by **2.3e-9** — small, deterministic, and invisible to any tolerance check. It is now zeroed per configuration, a no-op for a one-shot run.

Both were found only because the acceptance test is bit-identity rather than agreement. The first two attempts looked correct and were not.

5 End-to-end result: the 200 nm verification case

A 200 nm hexagonal march to 1 dpa (doses 0.01, 0.1, 1.0; 5 substeps per interval; `fem_every` = 5), measured against the *true* pre-optimization code — no sharding, 80 threads, pre-optimization DDomp:

	original	optimized	speedup
fast solve, 1st (builds everything)	28.1 s	21.5 s	1.31×
fast solve, steady state	28.1 s	12.1 s	2.32×
slow step (sum of three intervals)	67.0 s	28.0 s	2.39×
march wall	161.7 s	84.1 s	1.92×

`march.state.npz` is **bit-identical** to the original: max absolute difference 0.000×10^0 over $4 \times 21\,052 \times 19$ values.

An earlier baseline in this note quoted 1.22×; that baseline already contained sharding and the thread fix and so understated the total. The figure above is against code with every optimization removed.

6 The production case: 1000 nm

The 200 nm case is the *verification* measurement — small enough to run twice and diff the state array. It is not the case the optimization was for, and it understates the gain, because three of the five fixes remove costs that grow *faster* than the solve itself ($N^{1.86}$ for the Cholesky against $N^{1.33}$ for the mobile solve).

The production case is a 1000 nm×1600 nm hexagonal prism, **189 533 CD nodes**, 16 doses from 10^{-4} to 40 dpa, 20 substeps per interval, `fem_every` = 5 (320 substeps, 64 fast solves). It has been run twice to completion: once before this work, and once after.

mean over the 15 matched intervals	before	after	speedup
fast step (4 solves)	2040.8 s	798.1 s	2.56×
— per fast solve	510.2 s	199.5 s	2.56×
slow step (20 substeps)	1397.9 s	268.8 s	5.20×
interval wall	3438.6 s	1066.8 s	3.22×

Every matched interval does the same work: 4 fast solves, 20 substeps, dedup ×1.21 on both runs, no failed nodes and no retries on either. The first interval differs (3 fast solves) and is excluded from the mean; taken on its own it gives 2989.4 s → 925.2 s, 3.23×, the same figure.

whole march, 16 intervals	before	after	speedup
march wall, as reported	15.36 h	4.89 h	3.14×
sum of interval times	15.16 h	4.70 h	3.22×

Both rows, because they differ and the difference is real. The march wall exceeds the sum of its intervals by 0.19 h after and 0.20 h before — snapshot writes, checkpointing and the state array — which is per-snapshot work the optimization does not touch and which therefore dilutes the ratio slightly, $3.22\times$ to $3.14\times$. The 3.14 is the number a user experiences and is the one quoted in the abstract.

Written first as a projection, and it held. An earlier revision of this section was written with 11 of 16 intervals complete and projected the remainder at the matched-interval mean, giving 4.71 h. The completed run measures 4.70 h. Interval time is flat to $\pm 1\%$ across all 15 matched intervals, which is what made the extrapolation safe; it is recorded here because the projection is no longer load-bearing and the agreement is worth more than the estimate was.

Why the production gain exceeds the verification gain ($3.21\times$ against $1.92\times$). It is the same five fixes, and the difference is N . The per-solve fast cost falls $509.9\text{ s} \rightarrow 200.2\text{ s}$, and the profiling table of §4 accounts for it directly: 212.5 s of every pre-optimization invocation was rebuild — Cholesky, startup, mesh, trial functions — which the resident solver now pays *once for the whole march* instead of 64 times. At 200 nm that same rebuild is 5.9 s of 14.9 s. The removed term is the one growing as $N^{1.86}$, so the speedup itself grows with domain size, and the case that needed it most benefits most.

One confound, stated. The two 1000 nm runs also differ in `SOLVER['loop_model']` — 0 before, 1 after, following the change of default — so they are not a controlled A/B of the optimization alone. The confound is confined to the **slow** step, whose right-hand side differs between the two formulations at the same state width (19); its $5.19\times$ therefore carries some unknown model component. The fast step solves the same FE problem on the same mesh with the same node count and the same 4 solves per interval, and its $2.55\times$ is consistent to 10% with the *independently profiled* rebuild cost above. The controlled measurement is the 200 nm one, which is bit-identical; this section is a production result, and it is quoted as one.

7 What remains

1. **The unexplained in-solve residual** — 28.96 s at 1000 nm against 0.26 s at 200 nm, $N^{2.14}$: the steepest term in the table and entirely unidentified. It is now the obvious next profiling target.
2. **Mesh creation** — 11.4 s at 1000 nm, amortized by the resident solver but still paid once, and emitted as 129 569 progress lines.
3. **dedup_rtol** — 10^{-8} against an integrator `rtol` of 10^{-6} ; dedup measures $\times 1.21$. Loosening it cuts slow work proportionally, with error bounded by a tolerance already accepted. Untested, and the cheapest item here.
4. **Node count** — the largest cost driver: the fast solve scales $N^{1.33}$ and the boundary layer is 125 k of the 1000 nm mesh's 130 k tets. A modelling decision, not an optimization.
5. **Making mSolver iterative** — would remove the Cholesky outright rather than amortizing it, but changes the Newton loop's starting iterate and so cannot be bit-identical. Deliberately not taken.

`fem_every` is excluded on purpose: splitting error is first order in the fast-solve spacing, so time saved there comes out of the physics.

Reproducing

```
DISLOCLUSTER_SOLVER_SHARDS=1 # one solver process per batch (pre-Fix-1)
DISLOCLUSTER_DDUMP_THREADS=20 # fast-step thread count
DISLOCLUSTER_DDUMP_SERVER=0 # one DDump process per solve (pre-Fix-5)
```

Caveat. Every number is from one host. The processor-group limit is Windows-specific, the shard plan is sized from `os.cpu_count()`, and the fast-step thread optimum shifted between the two problem sizes measured. Single stage timings vary by up to $\sim 25\%$ here, so the per-phase tables support which phase dominates, not their third digit. The 200 nm figure is a controlled, bit-identical measurement; the 1000 nm figure is a production comparison carrying the `loop_model` confound noted in §6. Both runs are complete; nothing in that section is extrapolated.